

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 08 -

# REFATORAÇÃO DE PROJETO DS

---

## SUMÁRIO

|                                   |    |
|-----------------------------------|----|
| O QUE VEM POR AÍ? .....           | 3  |
| HANDS ON .....                    | 4  |
| SAIBA MAIS.....                   | 6  |
| MERCADO, CASES E TENDÊNCIAS ..... | 17 |
| O QUE VOCÊ VIU NESTA AULA? .....  | 18 |
| REFERÊNCIAS.....                  | 19 |

EMAN

## O QUE VEM POR AÍ?

Refatorar é melhorar o design interno do código sem alterar o comportamento observável. Em projetos de DS, isso significa sair de notebooks longos, com estados implícitos e “funções Deus” para módulos coesos, testáveis e tipados; sair de data wrangling duplicado para pipelines reutilizáveis; e transformar “provas de conceito” em artefatos evolutivos. Você aprenderá a diagnosticar bad smells (duplicações, funções gigantes, variáveis globais), aplicar refatorações clássicas (p. ex., Extract Function, Rename, Move Function) e reorganizar a arquitetura do projeto para escalar com segurança.

Ao longo da aula, vamos conectar teoria e prática: usamos testes automatizados (pytest) como guarda-chuva para garantir que o comportamento não mude; adotamos pipelines do scikit-learn para eliminar vazamento de dados; medimos custo temporal e de memória (NumPy/pandas) para decisões de desempenho; e fechamos com padrões de organização (pacotes Python com pyproject.toml) que favorecem manutenção, CI/CD e colaboração. Assista às videoaulas e revise os trechos de código sempre que avançar no texto.

## HANDS ON

Na videoaula prática, você acompanhará a transformação de um projeto legado “bagunçado” em um projeto modular e testável.

Pegaremos um código real de DS com problemas: duplicação em notebooks, uma função `train_and_plot` que treina um modelo e gera gráfico e salva arquivo (tudo misturado!), além de não haver testes. Nos vídeos, passo a passo, refatoraremos esse projeto. Vamos extrair funções de trechos repetidos, montar um pipeline do scikit-learn para separar etapas de pré-processamento e modelagem e escrever testes com PyTest para garantir que o comportamento permanece correto.

Em paralelo, organizamos a estrutura de pastas (criando um pacote pipeline/ com módulos) e configuramos um `pyproject.toml` para instalar o projeto em modo editável – assim, aprendemos a integrar engenharia de software ao fluxo de ciência de dados. Assista às videoaulas e tente reproduzir: pausar em cada etapa e executar no seu ambiente reforça o aprendizado. Os links usados nos vídeos para documentação (PyTest, Pipeline do scikit-learn, etc.) estão compilados no final desta seção, para você consultar.

Alguns dos códigos que iremos usar:

```
# Linguagem: Python
# arquivo: src/legacy_train.py (antes da refatoração)
import pandas as pd
from sklearn.linear_model import LogisticRegression

def train_and_plot(csv_path):
    df = pd.read_csv(csv_path)
    df = df.dropna()
    X = df[["x1", "x2", "x3"]]; y = df["target"]
    model = LogisticRegression(max_iter=200).fit(X, y)
    # ... vários efeitos colaterais (plots, gravação etc.)
    return model.score(X, y)
```

Código-fonte 1 – Função legada e teste de caracterização  
Fonte: Elaborado pelo autor (2025)

Explicação: a função `train_and_plot` lê dados CSV, faz pré-processamento, treina um modelo e ainda faz plot/salva resultados (indicados por comentário) – tudo junto. Isso é um típico “code smell”: a função faz mais do que deveria (violando o princípio de responsabilidade única) e mistura etapas que deveriam ser separadas.

Além disso, não há testes garantindo que o resultado (score) esteja correto. Vamos refatorar isso em etapas:

```
# Linguagem: Python
# arquivo: src/pipeline/model.py (depois)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

def build_pipeline():
    return Pipeline([
        ("scale", StandardScaler()),
        ("clf", LogisticRegression(max_iter=200))
    ])
```

Código-fonte 2 – Extração de funções e pipeline  
Fonte: Elaborado pelo autor (2025)

Explicação: aqui criamos uma função `build_pipeline` que monta um Pipeline do scikit-learn com duas etapas: primeiro escalonamento (`StandardScaler`) e depois o classificador (`LogisticRegression`). Esse pipeline integrará o pré-processamento e o modelo, garantindo que em produção façamos as mesmas transformações do treino. Agora podemos refatorar o código de treinamento para usar o pipeline, separando claramente a preparação de dados do treinamento em si.

```
# arquivo: tests/test_pipeline.py
import pandas as pd
from pipeline.model import build_pipeline

def test_pipeline_runs(tmp_path):
    df = pd.DataFrame({"x1": [0, 1], "x2": [1, 0], "x3": [0.5, 0.1], "target": [0, 1]})
    X, y = df[["x1", "x2", "x3"]], df["target"]
    pipe = build_pipeline()
    pipe.fit(X, y)
    assert pipe.score(X, y) >= 0.5
```

Código-fonte 3 – Teste com pytest e uso do pipeline  
Fonte: Elaborado pelo autor (2025)

Explicação: construímos um pequeno DataFrame de exemplo e testamos que nosso pipeline consegue treinar (`fit`) e obter pelo menos 50% de acurácia no próprio conjunto (que é mínimo, só para validar o fluxo). Testes como esse servem de rede de segurança – se quebrarmos algo na refatoração, o teste falha e sabemos que precisamos corrigir. Note que, antes de refatorar, ter um teste que captura o comportamento original (`model.score`) é essencial (teste de caracterização): assim garantimos que, após mudanças, o resultado continua igual.

## SAIBA MAIS

Nesta seção, vamos nos aprofundar tecnicamente nos conceitos e melhores práticas por trás da refatoração em projetos de ciência de dados. Prepare-se para uma jornada densa e didática: destrincharemos os code smells, técnicas de refatoração, testes automatizados, desempenho computacional e arquitetura de projetos, sempre com foco em Data Science. A ideia é consolidar seu entendimento de forma divertida e clara – conectando teoria com situações práticas de projetos de DS. Não deixe de pausar e refletir, e revisitar as videoaulas quando necessário, pois agora elevaremos o nível de detalhe. Vamos lá!

### Bad Smells em código de Data Science e refatorações fundamentais

O termo “code smell” (cheiro de código) refere-se a um indício superficial de que algo está errado no código. Em projetos de DS, alguns smells clássicos incluem: Código Duplicado (a mesma lógica copiada em vários notebooks ou scripts), Funções Muito Longas (fazendo mil coisas – e.g., limpeza, treino e plot – tudo em uma só), Variáveis Globais Excessivas (dependências implícitas difíceis de rastrear), Nomes Crípticos (df2, temp, etc., que não revelam propósito), entre outros.

Esses sintomas geralmente apontam problemas mais profundos de design. Por exemplo, código duplicado sugere falta de abstração (deveria ser uma função reutilizável), funções gigantes violam o princípio da responsabilidade única e uso de globais indica acoplamento oculto entre partes do código.

Martin Fowler popularizou muitos desses conceitos e catalogou técnicas de refatoração para atacá-los. Duas características importantes da refatoração: (a) é feita em pequenos passos – você transforma o código gradualmente, rodando testes a cada passo para garantir que o comportamento permanece igual; (b) foca na qualidade interna do código, não em adicionar funcionalidades novas. Em outras palavras, refatorar é “arrumar a casa” do código para facilitar evoluções futuras, sem mudar o que o software faz.

## Exemplos de Smells e Refatorações Possíveis

- Duplicação de código: se você notar o mesmo bloco de limpeza de dados em três notebooks, é hora de extrair essa lógica para uma função em um módulo comum, que todos podem importar. Aplica-se Extract Function ou Extract Module.
- Função “Deus” (God Function): uma função de centenas de linhas que faz de tudo. Refatore dividindo em funções menores (cada uma com responsabilidade específica) e chame-as em sequência. (Split Function, Extract Function repetidamente).
- Nomes ruins: se você vê variáveis a, b, c e não sabe o que são, renomeie-as para algo significativo (Rename Variable/Method). Às vezes adicionar um comentário pode ajudar, mas Fowler brinca que “comentários muitas vezes mask code smells – prefira refatorar para que o comentário não seja necessário”.
- Variáveis globais: passe parâmetros explícitos nas funções ao invés de depender de globais ou encapsule em classes. Isso aumenta a clareza de dependências (Encapsulate Variable).
- Código morto: remove partes não usadas (Delete Dead Code); menos código, menos bugs.

Ferramentas modernas ajudam a detectar alguns smells. Linters como flake8 podem apontar funções muito extensas ou variáveis não usadas. Mas muitas detecções vêm da review manual mesmo – com experiência, você “sente” o cheiro ruim no código (Kent Beck dizia que seu nariz coçava ao ver uma função muito longa). O importante é: detectou um smell, aplique uma refatoração. Faça isso iterativamente e seu projeto evolui de forma limpa.

Vale a leitura do guia de Ben Feifke, focado em refatoração para cientistas de dados. Ele mostra casos comuns: junções de notebooks em scripts reutilizáveis, generalização de funções para múltiplos datasets etc., tudo pensado para quem não é engenheiro(a) de software de formação.

Resumindo: refatorar não é opcional se você quer que seu código de data science sobreviva além do primeiro resultado. Pequenas melhorias contínuas evitam a famosa dívida técnica de crescer tanto que o projeto precisa ser refeito do zero.

| SMELL (SINTOMA)                               | PROBLEMAS QUE CAUSA                           | POSSÍVEL REFATORAÇÃO                                                                                                   |
|-----------------------------------------------|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Duplicação de código em vários locais         | Inconsistência; mais locais para corrigir bug | Extract Module: centralizar em um módulo/função reutilizável                                                           |
| Função monolítica faz "tudo"                  | Difícil de testar; difícil de reutilizar      | Extract Function: quebrar em funções menores; Split Phase: separar etapas (ex: preparar dados vs treinar modelo)       |
| Variáveis globais usadas amplamente           | Fluxo implícito, ordem de execução rígida     | Encapsulate Variable: torná-las atributos de classe ou parâmetros de função                                            |
| Nomes pouco descritivos (data1, foo)          | Código obscuro; propensão a erros             | Rename: renomear para refletir propósito (ex: clientes_df, calcular_receita())                                         |
| Comentários excessivos explicando "gambiarra" | Indicam código confuso ou duplicado           | Refatore para eliminar a gambiarra; usar comentários apenas para contexto adicional, não para explicar lógica tortuosa |

Tabela 1 – Exemplos de Code Smells comuns e estratégias de Refatoração  
Fonte: Elaborado pelo autor (2025)

Refatoração segura com testes automatizados e pipelines

Refatorar só é seguro quando temos testes automatizados cobrindo o comportamento atual. Em projetos de DS, muitas vezes faltam testes – aí a



refatoração vira um tiro no escuro. Por isso, recomendamos fortemente: antes de modificar código crítico, escreva alguns testes de caracterização. Eles capturam o output atual dado um input conhecido, servindo de contrato. No exemplo do projeto legado, escrevemos um teste que treina o modelo e verifica a acurácia (Código-fonte 3) – assim sabemos o número esperado. Quando refatorarmos, se esse número mudar sem motivo, o teste falha e sinaliza problema. Testes permitem confiança ao melhorar código. [\[Atualizaçã...jamento de | Word\]](#)

Além disso, em DS, podemos aproveitar frameworks como PyTest para facilitar. Ele permite parametrizar testes com diferentes dados, criar fixtures (por ex., um DataFrame de exemplo disponível para vários testes) e integrar com relatório de cobertura de código. Um coverage alto (digamos, >80%) indica que grande parte do código é exercitada por testes – não é uma meta absoluta, mas ajuda.

Uma estratégia esperta: ao detectar um bug ou discrepância antes da refatoração, escreva primeiro um teste que falhe evidenciando o bug. Em seguida, arrume o código (refatore ou corrija). Assim você garante que a correção está coberta e não volta atrás.

Pipelines e testes em DS: o scikit-learn implementa pipelines justamente para encadear transformações e modelos de forma testável e sem vazamentos de dados. Se você refatora código de pré-processamento espalhado em várias partes para um pipeline único, fica fácil testar a pipeline inteira de uma vez (como fizemos). Outra vantagem: pipelines podem ter seu output comparado antes e depois da refatoração, garantindo que nada mudou. Em resumo, use ferramentas do ecossistema para ajudar na refatoração. Integrar com CI (Continuous Integration) – e.g., GitHub Actions rodando testes a cada push – automatiza essa segurança: qualquer refatoração sobe somente se os testes passarem em todos os cenários.

## **Desempenho, memória e boas práticas computacionais ao refatorar**

Refatorar não é só deixar o código “bonito” – às vezes revelamos problemas de desempenho escondidos. Por exemplo, imagine que ao separar uma função de limpeza, você perceba que ela faz um loop Python ineficiente sobre milhares de linhas. Ao isolá-la, fica mais claro que uma vetorização com NumPy ou pandas seria melhor. Aproveite a refatoração para melhorar eficiência, mas sem mudar o

resultado – faça em passos: primeiro garanta a correção, depois optimize se necessário, validando que resultados permanecem iguais (teste de regressão de performance).

Dois conceitos importantes em Python para performance: broadcasting do NumPy e distinção entre views e copies em pandas.

- **Broadcasting:** permite aplicar operações em arrays de tamanhos diferentes, expandindo o menor, de forma vetorizada. Exemplo: somar um vetor de médias a cada linha de uma matriz – evite loops Python e use arrays NumPy; o NumPy “espalha” (broadcast) o vetor nas dimensões da matriz, executando em C, muito mais rápido. Em refatorações, busque trocar loops explícitos por operações vetorizadas. Mas cuidado: broadcasting descontrolado pode consumir memória demais (ele expande dados implicitamente). Sempre teste com volumes maiores para ver se a otimização não virou um gargalo de memória.
- **Views vs Copies:** em Pandas, ao fatiar um DataFrame, às vezes obtemos uma view (referência aos mesmos dados em memória) e outras uma copy (dados copiados). Operar sobre views pode ter efeitos colaterais inesperados no DataFrame original, enquanto as copies consomem mais RAM. A comunidade pandas introduziu recentemente o copy-on-write (a partir do pandas 2.1) para tornar isso mais previsível. Durante refatorações, tente escrever código explícito – por exemplo, use `df.copy()` quando realmente quer duplicar dados para não afetar o original. E ao encadear operações pandas, fique de olho nos `SettingWithCopyWarning`, sinal de ambiguidade view/copy. Um código refatorado ideal deixa essa questão clara, evitando bugs sutis de dados compartilhados ou gasto de memória duplicado.

**Complexidade algorítmica:** ao modularizar, pense também: “Qual é a ordem de complexidade dessa função?”. Uma função limpando dados registro a registro é  $O(n)$ ; se dentro dela você colocou uma busca linear em uma lista para cada registro, sem perceber introduziu  $O(n^2)$ .

Na aula de hoje não entraremos em detalhes matemáticos de complexidade, mas lembre-se: faz parte da engenharia de software pensar se sua refatoração

mantém o código escalável. Se você notar um bottleneck, talvez seja hora de introduzir uma estrutura de dados melhor (como usar um set/dict em vez de lista para busca, reduzindo de  $O(n)$  para  $O(1)$  amortizado). Refatorar é uma chance de também otimizar algoritmos, contanto que você teste que o resultado final do código não mudou.

Em resumo, refatoração em DS também aborda performance: código limpo frequentemente é código mais eficiente, mas fique de olho para não piorar sem querer – sempre valide tempo de execução e uso de memória com amostras representativas.

### Reestruturando a arquitetura do projeto: do caos à modularidade

Além de melhorar funções e arquivos individuais, a refatoração pode envolver reorganizar todo o projeto. Projetos de ciência de dados muitas vezes começam com uma sequência de notebooks (analise1.ipynb, analise2.ipynb...) e talvez uns scripts soltos. Com o tempo, isso vira uma colcha de retalhos difícil de manter. Refatorar arquitetura significa adotar uma estrutura de pastas e módulos coesa e escalável.

Uma prática recomendada é passar para um formato de pacote Python instalável. Por exemplo, criar esta estrutura:

```
project_name/  
├── src/  
│   ├── data/  
│   ├── models/  
│   ├── utils/  
│   └── __init__.py  
├── tests/  
├── pyproject.toml  
└── README.md
```

Figura 1 – Exemplo de estrutura de pastas  
Fonte: Elaborado pelo autor (2025)

Dentro de `src/`, separamos subpacotes por responsabilidade: `data` para código de obtenção e limpeza de dados, `models` para treinamento e avaliação de modelos, `utils` para funções auxiliares etc. Isso força uma modularização: em vez de um notebook carregar tudo, cada componente vira um módulo importável. Os notebooks existentes podem ser refatorados para simplesmente chamar funções desses módulos – tornando-os mais leves, quase como relatórios reprodutíveis, enquanto a lógica fica no pacote.

O arquivo `pyproject.toml` configura o projeto como pacote instalável. Nele definimos dependências, versão de Python requerida e até entry points (como scripts de linha de comando úteis). Com isso, podemos instalar localmente (`pip install -e .`) e usar em outros projetos, ou integrar com pipelines CI/CD facilmente. A seguir um exemplo mínimo:

```
[project]
name = "ds_refatorado"
version = "0.1.0"
dependencies = [
    "pandas>=2.1",
    "scikit-learn>=1.4",
    "pytest>=8.0"
]
[project.scripts]
treinar-modelo = "src.models.train:main"
```

Código-fonte 4 – Trecho de `pyproject.toml` configurando pacote e script  
Fonte: Elaborado pelo autor (2025)

Explicação: definimos nome e versão do projeto, dependências (`pandas`, `sklearn`, `pytest`, etc.) e um script chamado `treinar-modelo` que apontará para a função `main` em `src/models/train.py` (hipotético). Assim, após instalar, o usuário poderia executar `treinar-modelo` no terminal para rodar o pipeline de treinamento completo, por exemplo. Isso ilustra como um projeto DS refatorado aproxima-se de um software convencional, com possibilidade de distribuição e reuso.

Organizar o projeto dessa forma traz enormes ganhos:

- **Manutenibilidade:** localizar código fica fácil (“Quero ver como lida com dados? Olhe em `src/data`”). Novos membros da equipe entendem a estrutura rapidamente.
- **Modularidade:** podemos evoluir cada parte isoladamente (por exemplo, refazer toda a lógica de `src/data` sem tocar em `src/models`, contanto que

as interfaces – inputs/outputs – se mantenham). Isso é arquitetura de baixo acoplamento na prática.

- **Testabilidade:** facilita escrever testes organizados por módulos (teste para data, teste para models etc.).
- **Integração contínua:** fica mais simples automatizar lint, testes e builds, pois temos padrões convencionais de projeto.

É claro que nem todo projeto pequeno precisará virar um pacote formal. Mas se o código de um case interno for evoluir para algo reutilizável ou de longo prazo, essa reestruturação arquitetural é um investimento que compensa. E você pode fazer essa transição incrementalmente: comece movendo algumas funções utilitárias para um módulo, depois outras, até que grande parte da lógica esteja fora dos notebooks. Aos poucos, seu repositório se parecerá mais com um projeto de software bem organizado, em que notebooks são apenas consumidores ou visualizadores de resultados.

**Dica:** consulte o livro “Clean Code in Python” (J. Aniche, 2021) ou “Clean Code in Data Science” (existe material emergente nessa linha) para ver exemplos de estruturação modular em projetos de ML. A O’Reilly também tem um título Clean Code in Machine Learning que aborda arquitetura de projetos de ML – vale a referência. Esses recursos mostram casos reais de passar de scripts desorganizados para pacotes coesos.

## Pipelines e separação de responsabilidades

Como parte da arquitetura, vale reforçar o uso de pipelines de processamento e modelagem. Já introduzimos anteriormente com scikit-learn Pipeline (Código 2 e 3). A separação de responsabilidades também ocorre temporalmente: geralmente dividimos um projeto DS em etapas como ingestão de dados, limpeza, feature engineering, treinamento, validação, deploy. Cada etapa pode (e deve) ser uma unidade independente.

Em refatoração, um padrão útil é Split Phase – separar fases de um processo que antes estavam misturadas em uma função ou script. Por exemplo, se um único notebook faz desde consultas SQL até avaliar modelo, crie um módulo `data_ingestion` para as consultas, outro `model_training` para o modelo etc. Isso não só ajuda na organização, mas permite eventualmente rodar esses passos

separadamente ou até em ambientes distintos (ex.: puxar dados em um servidor de banco, treinar no computador com GPU...).

O Scikit-learn Pipeline nos dá um encapsulamento dessa separação dentro de um mesmo objeto de modelagem. Ele evita data leakage (vazamento de dados) ao garantir que transformações de dados durante o treino se apliquem da mesma forma no teste e que nada do futuro contamine o passado. Nosso exemplo garantiu que o StandardScaler é fitado só com X de treino e depois aplicado – tudo automaticamente dentro do `pipe.fit()` e `pipe.predict()`. Em termos de refatoração, isso substitui código manual de pré-processamento + treino, reduzindo potencial de erro. E se quisermos testar o pipeline inteiro, fica uma linha.

Além disso, pipelines facilitam grid search conjunto – você pode variar hiperparâmetros tanto do modelo quanto das etapas de pré-processamento de forma integrada (e.g., otimizar ou não a normalização). Ou seja, há ganhos de manutenção e também de capacidade analítica.

### O dilema dos notebooks: como “domar” e integrar na engenharia

Muitos cientistas de dados adoram notebooks (Jupyter, Colab) pela interatividade. Mas notebooks são notórios por estimularem bad practices: execução fora de ordem, estados implícitos, dificuldade de versionar. O que fazer? Refatoração aqui significa adotar disciplina no uso de notebooks:

- **Modularize dentro do notebook:** use funções e classes mesmo no notebook, não deixe tudo em células soltas. Assim, se o notebook virar um script, o código já está estruturado.
- **Restart and Run All:** habitue-se a sempre reexecutar do zero o notebook completo para garantir reprodutibilidade e pegar dependências ocultas.
- **Orquestre notebooks externamente:** ferramentas como Papermill ou nbconvert permitem parametrizar e rodar notebooks via linha de comando (úteis para incorporar notebooks em pipelines automatizados).
- **Versione notebooks com cuidado:** commits de notebooks podem ser ruidosos (metadados, outputs). Use ferramentas como nbstripout para limpar antes de versionar e revise manualmente quando possível.

Documente o propósito de cada notebook (ex.: “Notebook X explora correlação, Notebook Y faz modelagem final”).

- **Migrar para módulos quando estiver estável:** a abordagem comum é: explorou e validou no notebook, agora transforme em módulo .py para produção. Isso é uma forma de refatoração maior. Existem utilitários como jupyter-to-script que extrai código de notebooks marcados com tags especiais. Mas muitas vezes, é reescrever manualmente mesmo – aproveitando para melhorar a estrutura.

Um caso interessante é o do Databricks: eles permitem notebooks em produção, mas recomendam um conjunto de melhores práticas (como desenvolver em repos Git integrados, testar notebooks com %run, utilizar separação de configuração etc.). É um exemplo de adaptação: se notebooks são inevitáveis, aplique engenharia neles. Outro exemplo é o Netflix Metaflow e frameworks similares (Papermill, MLflow pipelines) que ajudam a transformar experimentos exploratórios em flows reproduzíveis e versionados.

Em suma, notebooks não precisam ser inimigos da engenharia, mas requerem vigilância constante. Como Fowler diria, “refatoração constante” – no caso, passe o pente-fino no seu notebook regularmente para deixá-lo limpo e dividir partes em funções.

### **Dívida técnica em sistemas de ML: refatorar além do código**

Refatoração também se aplica a componentes “não código” de projetos de Machine Learning. O famoso artigo do Google “Hidden Technical Debt in ML Systems” mostra que a maioria do sistema de ML é infraestrutura: dados, configuração, monitoramento, etc., não apenas o código do modelo. Smells podem existir nessas áreas também: conjuntos de dados duplicados, falta de monitoramento de performance do modelo, scripts de deploy manuais e não reproduzíveis etc. A engenharia de software nos ensina a refatorar esses processos: normalizar um pipeline de dados duplicado, consolidar fontes de verdade de features, automatizar implantações com infraestrutura como código e assim por diante.

Um caso real dramático: o Zillow Offers (um serviço de precificação imobiliária via ML) fracassou em 2021, gerando enormes prejuízos. Análises posteriores indicam que parte do problema foi não ter sistemas robustos para monitorar e ajustar o modelo conforme o mercado mudava, acumulando uma espécie de “dívida técnica” de ML – confiou-se cegamente no modelo sem a engenharia em torno dele. Resultado: quando os dados mudaram drasticamente (pandemia), o modelo cometeu erros graves e a empresa não conseguiu reagir a tempo. Refatorar sistemas de ML inclui planejar desde cedo feedback loops e atualizações, ou seja, sistemas adaptáveis.

Portanto, pense na refatoração de projeto DS em camadas: comece pelo código (que é o foco desta aula), mas lembre-se que acima disso há processos e sistemas que também precisam de melhorias constantes. A boa notícia é que, ao estruturar bem o código e os dados, você torna esses níveis superiores muito mais fáceis de ajustar.



## MERCADO, CASES E TENDÊNCIAS

**Sucesso com plataforma e refatoração de fluxo** — o Metaflow (Netflix) nasceu para aproximar pesquisa e produção, formalizando flows e versão de artefatos. É exemplo de “refatoração de processo”: padronizar estrutura, declarar dependências e permitir scale-out sem reescrever tudo. Leitura recomendada: docs e techblog (casos recentes de evolução). Assista também a talks abertas sobre flows e sandbox. [\[docs.metaflow.org\]](https://docs.metaflow.org/), [\[netflixtechblog.com\]](https://netflixtechblog.com/)

**Conhecimento reprodutível** — Airbnb Knowledge Repo institucionaliza “posts reprodutíveis” (inclui notebooks e metadados) e reduz a duplicação de análises, que é um smell organizacional. Um bom exemplo de refatoração **organizacional** a partir da prática. [\[airbnb.io\]](https://airbnb.io/), [\[knowledge-...thedocs.io\]](https://knowledge-...thedocs.io/)

**Alertas de fracasso (quando não refatorar dói)** — O caso **Zillow Offers** ilustra como pipelines frágeis e governança insuficiente amplificam riscos de concept drift e decisões de alto impacto sem guardrails. Post-mortems públicos e material acadêmico ajudam a discutir o que deveria ter sido refatorado (monitoramento, limites operacionais, rollback). Use em sala para debate crítico. [\[gsb.stanford.edu\]](https://gsb.stanford.edu/), [\[cnet.com\]](https://cnet.com/)

### Ferramentas e vídeos abertos para aprofundar

- [PyPA](https://pyproject.toml): pyproject.toml e Real Python — empacotamento moderno. **Refaça o seu projeto** durante a semana.
- [Refactoring catalog](#) (Martin Fowler) — web aberta, com técnicas e exemplos.
- [I Don't Like Notebooks](#) — Joel Grus (provocações úteis para migrar lógica para módulos). [Assista](#) e traga contrapontos.
- [Databricks](#) — best practices para notebooks — guia passo a passo para versionar, testar e orquestrar. Experimente reproduzir no seu workspace.

## O QUE VOCÊ VIU NESTA AULA?

Você praticou diagnóstico de smells em projetos de DS, aplicou refatorações incrementais guiadas por testes e reorganizou um projeto em pacote Python com pipeline e tipagem.

Discutimos custo computacional (vetorização, broadcasting, views/copies), arquitetura modular e orquestração responsável de notebooks.

Como CTA, assista novamente às videoaulas, execute os testes localmente e refatore uma etapa extra do seu projeto (ex.: extraia mais uma função e adicione um novo teste).

## REFERÊNCIAS

AIRBNB. **Airbnb – Knowledge Repo**. 2025. Disponível em: <https://airbnb.io/projects/knowledge-repo/>. Acesso em: 16 dez. 2025.

AIRBNB. **Knowledge Repo v0.9.3 documentation**. 2025. Disponível em: <https://knowledge-repo.readthedocs.io/en/latest/>. Acesso em: 16 dez. 2025.

DATABRICKS. **Software engineering best practices for notebooks**. 2025. Disponível em: <https://docs.databricks.com/aws/en/notebooks/best-practices>. Acesso em: 16 dez. 2025.

FEIFKE, B. **Refactoring for Data Scientists: A Beginner's Guide**. 2023. Disponível em: <https://benfeifke.com/posts/refactoring-for-data-scientists-a-beginners-guide/>. Acesso em: 16 dez. 2025.

FOWLER, M. **Refactoring – Catalog of Refactorings**. 2025. Disponível em: <https://refactoring.com/catalog/>. Acesso em: 16 dez. 2025.

GRUS, J. **I Don't Like Notebooks**. 2018. Disponível em: <https://www.youtube.com/watch?v=7jiPeIFXb6U>. Acesso em: 16 dez. 2025.

NETFLIX. **Metaflow – Docs**. s.d. Disponível em: <https://docs.metaflow.org/>. Acesso em: 16 dez. 2025.

NUMPY. **NumPy – Broadcasting**. 2025. Disponível em: <https://numpy.org/doc/stable/user/basics.broadcasting.html>. Acesso em: 16 dez. 2025.

PANDAS. **pandas docs (2.3.x)**. 2025. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 16 dez. 2025.

PYPA. **Writing your pyproject.toml**. 2025. Disponível em: <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>. Acesso em: 16 dez. 2025.

PYTEST. **Get Started – pytest**. 2025. Disponível em: <https://docs.pytest.org/en/stable/getting-started.html>. Acesso em: 16 dez. 2025.

SCIKIT-LEARN. **Pipeline — scikit-learn**. 2025. Disponível em: <https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>. Acesso em: 16 dez. 2025.

SCULLEY, D. et al. **Hidden Technical Debt in Machine Learning Systems**. 2015. Disponível em:

<https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf>. Acesso em: 16 dez. 2025.

EMSE

## **PALAVRAS-CHAVE**

Refatoração incremental; Pipelines de ML; Arquitetura modular em Data Science.

EMSE



POSTECH